NOIP 模拟卷

第二套

提高级

题目名称	连续检修	协同巡检	三机编排	区间合码
题目类型	传统型	传统型	传统型	传统型
目录	repair	patrol	compile	orcost
可执行文件名	repair	patrol	compile	orcost
输入文件名	repair.in	patrol.in	compile.in	orcost.in
输出文件名	repair.out	patrol.out	compile.out	orcost.out
每个测试点时限	2 秒	3 秒	3 秒	2 秒
内存限制	256 MiB	512 MiB	256 MiB	256 MiB
测试点数目	20	20	20	20
测试点是否等分	是，每点 5 分	是，每点 5 分	是，每点 5 分	是，每点 5 分

提交源程序文件名

对于 C++ 语言	repair.cpp	patrol.cpp	compile.cpp	orcost.cpp

编译选项

对于 C++ 语言	-O2 -std=c++14

注意事项（请仔细阅读）

1. 使用 C++14 编译，编译选项为 -O2 -std=c++14。
2. 各题采用表中指定的输入、输出文件，文件名区分大小写。
3. 每题共有 20 个测试点，每点 5 分，独立计分；整套满分为 400 分。
4. 第三题包含多组数据，须同时满足单组限制和所有组的总量限制。
5. 附加样例位于对应题目的下发目录；PDF 中的样例行号不属于实际输入输出。

连续检修（repair）

【题目描述】

一条生产线上有 n 台设备，从左到右编号为 $1, 2, \ldots, n$。检修第 i 台设备时，工具的强度必须不小于 a_i。

你需要确定工具的初始强度 S，并选择两个不同的设备作为起点。这两台设备同时完成检修，此时工具强度均为 S，不计入后续检修的步数。

随后，现场调度员按照以下规则检修剩下的设备：

- 每一步选择一台尚未检修、且与至少一台已检修设备相邻的设备；
- 第 t 步使用的工具强度为 $S - t$，其中 t 从 1 开始；
- 若工具强度小于所选设备的要求，检修失败。

两个起点由你选择，后续顺序由调度员决定。你要保证：对于调度员的任意合法选择顺序，所有设备都能完成检修。

求满足要求的最小初始强度 S。

【输入格式】

从文件 **repair.in** 中读入数据。

第一行包含一个整数 n。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$。

【输出格式】

输出到文件 **repair.out** 中。

输出一个整数，表示最小初始强度。

【样例 1 输入】

```
4
1 8 8 1
```

【样例 1 输出】

```
8
```

【样例 1 解释】

选择第 $2, 3$ 台设备作为起点。剩下两台设备的要求均为 1，无论先检修哪台都能完成。由于有设备要求为 8，初始强度不能更小。

【样例 2 输入】

```
5
10 1 9 1 10
```

【样例 2 输出】

```
12
```

【样例 2 解释】

选择第 $1,5$ 台设备作为起点。调度员可以先检修第 $2,4$ 台，再在第 3 步检修第 3 台，因此初始强度至少需要 $9+3=12$。若没有把两个端点都选作起点，未选中的端点可以留到第 3 步，其要求为 10，需要的强度反而更大。

【样例 3 输入】

```
2
5 12
```

【样例 3 输出】

```
12
```

【样例 4】

见下发文件中的 **repair/repair4.in** 与 **repair/repair4.ans**。该样例的所有设备要求相同。

【样例 5】

见下发文件中的 **repair/repair5.in** 与 **repair/repair5.ans**。

【样例 6】

见下发文件中的 **repair/repair6.in** 与 **repair/repair6.ans**。

【样例 7】

见下发文件中的 **repair/repair7.in** 与 **repair/repair7.ans**。

【样例 8】

见下发文件中的 **repair/repair8.in** 与 **repair/repair8.ans**。

【数据范围】

每个测试点为 5 分，各测试点独立计分。

测试点编号	n 的上限
$1\sim 2$	9
$3\sim 5$	100
$6\sim 10$	5000
$11\sim 20$	300000

对于全部数据，$2\le n\le 300000$，$1\le a_i\le 10^9$。

协同巡检（patrol）

【题目描述】

一片园区有 n 个站点，编号为 $1 \sim n$，由 $n-1$ 条双向道路连成一棵树。每条道路的长度均为 1。两个站点之间的距离，是它们之间唯一路径上的道路条数。

巡检设备 A 和 B 最初都在站点 1。每一步，你可以选择其中一台设备，让它沿一条道路移动到相邻站点，另一台设备保持不动。整个过程中，两台设备之间的距离都不能超过给定的 d。两台设备可以位于同一个站点。

设备 A 和 B 各有一份必须访问的站点清单。每台设备须至少到达自己清单中的每个站点一次，访问顺序任意；设备之间不能代替对方完成任务。若清单包含站点 1，该项任务在开始时便已完成。两份清单可以有相同的站点。

当两台设备各自完成全部任务后，巡检即可结束，**不要求返回站点 1，也不要求停在同一个站点**。已经完成自己任务的设备仍然可以移动。

请计算完成巡检所需的最少总移动步数。

【输入格式】

从文件 **patrol.in** 中读入数据。

第一行包含两个整数 n, d。

接下来 $n-1$ 行，每行包含两个整数 u, v，表示站点 u 与 v 之间有一条道路。

接下来一行包含一个整数 m_A，表示设备 A 的任务数量。

接下来一行包含 m_A 个互不相同的整数，表示设备 A 必须访问的站点。

接下来一行包含一个整数 m_B，表示设备 B 的任务数量。

接下来一行包含 m_B 个互不相同的整数，表示设备 B 必须访问的站点。

【输出格式】

输出到文件 **patrol.out** 中。

输出一行一个整数，表示最少总移动步数。

【样例 1 输入】

```
3 2
1 2
1 3
1
2
1
3
```

【样例 1 输出】

```
2
```

【样例 1 解释】

先让 A 从站点 1 移动到站点 2，再让 B 从站点 1 移动到站点 3。两台设备的距离依次为 $1, 2$，共移动 2 步后即可结束。

【样例 2 输入】

```
5 2
1 2
2 3
1 4
4 5
2
3 5
2
3 5
```

【样例 2 输出】

```
12
```

【样例 2 解释】

两台设备可以都沿 $1 \to 2 \to 3 \to 2 \to 1 \to 4 \to 5$ 巡检。经过每条道路时，先移动一台，再移动另一台，便能保持距离不超过 1。每台设备移动 6 步，最后都停在站点 5。

【样例 3 输入】

```
9 2
1 2
2 3
3 4
4 5
1 6
6 7
7 8
8 9
2
5 9
1
1
```

【样例 3 输出】

```
18
```

【样例 3 解释】

B 虽然在开始时就完成了任务，仍须移动以保持两台设备的距离。可以先让 A 到达站点 5、B 到达站点 3，再让两台设备返回站点 1；接着让 A 到达站点 9、B 到达站点 7 并结束。每段路途中合理交替移动，总计需要 18 步。

【样例 4】

见下发文件 **patrol/patrol4.in** 与 **patrol/patrol4.ans**。该样例满足 $n = 80, d = 3$。

【样例 5】

见下发文件 **patrol/patrol5.in** 与 **patrol/patrol5.ans**。该样例满足 $n = 2000$。

【样例 6】

见下发文件 **patrol/patrol6.in** 与 **patrol/patrol6.ans**。该样例满足 $n = 200000$，且每个站点连接的道路不超过 3 条。

【样例 7】

见下发文件 **patrol/patrol7.in** 与 **patrol/patrol7.ans**。该样例满足 $n = d = 200000$。

【样例 8】

见下发文件 **patrol/patrol8.in** 与 **patrol/patrol8.ans**。该样例满足 $n = 200000$，没有其他特殊限制。

【数据范围】

本题共有 20 个测试点，每个测试点 5 分。各行表示对应测试点的保证；不同测试点分别计分。

测试点	分值	n 的上限	特殊性质
$1 \sim 2$	10	7	无
$3 \sim 5$	15	80	无
$6 \sim 8$	15	3000	无
$9 \sim 10$	10	200000	每个站点连接的道路不超过 3 条
$11 \sim 12$	10	200000	$d = n$
$13 \sim 20$	40	200000	无

对于全部数据，保证 $2 \le d \le n \le 200000$，$1 \le m_A, m_B \le n$，所有站点编号均在 $1 \sim n$ 内，给出的道路构成一棵树，每份任务清单内部没有重复站点。

三机编排（compile）

【题目描述】

一间工坊有三台相同的加工机，准备依次完成 n 项任务。每项任务属于 $1 \sim k$ 中的一种类型，第 i 项任务的类型为 a_i。

每台机器会保留自己上一次加工所用的工装。把一项类型为 x 的任务交给某台机器时：

- 如果这台机器上一次完成的任务也是类型 x，本次需要 h_x 秒；
- 如果这台机器尚未处理过任务，或上一次处理的是其他类型，本次需要 c_x 秒。

已知 $1 \le h_x \le c_x$。机器空闲多久都不会改变保留的工装，其他机器处理任务也不会影响它。

任务必须严格按给定顺序完成：第 i 项任务结束后，才能开始第 $i+1$ 项任务，不能同时加工。每项任务可以任选一台机器。三台机器最初都没有处理过任务。

请安排每项任务使用哪台机器，使完成全部任务所需的总时间最少。

【输入格式】

从文件 **compile.in** 中读入数据。

第一行包含一个整数 t，表示测试用例数量。每个测试用例包含四行：

第一行包含两个整数 n, k。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$。

第三行包含 k 个整数 $c_1, c_2, \ldots, c_k$。

第四行包含 k 个整数 $h_1, h_2, \ldots, h_k$。

不同测试用例相互独立，机器的工装不会保留到下一个测试用例。

【输出格式】

输出到文件 **compile.out** 中。

对每个测试用例输出一行一个整数，表示完成全部任务所需的最少总时间。

【样例 1 输入】

```
1
6 3
1 2 3 1 2 3
9 5 4
1 2 1
```

【样例 1 输出】

```
22
```

【样例 1 解释】

三台机器分别负责一种类型。每种类型第一次出现时支付冷启动费用，第二次出现时支付热启动费用，总用时为 $9 + 5 + 4 + 1 + 2 + 1 = 22$ 秒。任何安排中，每种类型的首次加工都不能使用热启动费用，因而这一安排最优。

【样例 2 输入】

```
1
8 4
1 2 3 4 1 2 3 4
10 8 7 6
1 2 3 4
```

【样例 2 输出】

```
45
```

【样例 2 解释】

第一台机器处理第 $1,5,7$ 项任务，第二台处理第 $2,6$ 项，第三台处理第 $3,4,8$ 项。用时依次为 $10,8,7,6,1,2,7,4$ 秒，总计 45 秒。

如果所有类型再次出现时都能热启动，总用时应为 41 秒。但完成第 4 项任务后，三台机器中至少一台保留类型 4，类型 $1,2,3$ 中至少有一种工装未被保留。这种类型下次出现时必须冷启动，至少多用 $\min(10-1,8-2,7-3)=4$ 秒，所以答案不小于 45。

【样例 3 输入】

```
3
1 5
5
5 7 11 12 13
1 7 2 3 8
4 1
1 1 1 1
10
3
3 3
1 2 3
5 5 5
5 5 5
```

【样例 3 输出】

```
13
19
15
```

【样例 3 解释】

第一组只处理一项任务，初次加工需要 13 秒。第二组连续处理同一类型，使用同一台机器，用时 $10+3+3+3=19$ 秒。第三组所有冷、热启动费用均为 5，三项任务共需 15 秒。

【样例 4】

见附件 **compile/compile4.in** 与 **compile/compile4.ans**。该样例满足 $n=k=200$。

【样例 5】

见附件 **compile/compile5.in** 与 **compile/compile5.ans**。该样例达到 n,k 的上界，包含多轮不同顺序的任务。

【样例 6】

见附件 **compile/compile6.in** 与 **compile/compile6.ans**。该样例达到 n, k 的上界，包含相邻的同类任务。

【样例 7】

见附件 **compile/compile7.in** 与 **compile/compile7.ans**。该样例达到 n, k 的上界，实际出现的任务类型不超过 3 种。

【样例 8】

见附件 **compile/compile8.in** 与 **compile/compile8.ans**。该样例达到 $\sum n, \sum k$ 的上界，每组实际出现的任务类型不超过 8 种。

【数据范围】

本题共 20 个测试点，每点 5 分，各测试点独立计分。表中的求和均对同一输入文件内的全部测试用例进行。令 C 表示一个测试用例的任务序列中实际出现的不同类型数。

测试点编号	分值	额外保证
$1 \sim 3$	15 分	$\sum n \le 11$
$4 \sim 7$	20 分	$\sum n \le 200$，且 $\sum k \le 200$
$8 \sim 10$	15 分	每个测试用例均满足 $C \le 3$
$11 \sim 13$	15 分	每个测试用例均满足 $C \le 8$
$14 \sim 20$	35 分	无额外保证

对于全部数据，保证：

$$1 \le t \le 1200, \qquad 1 \le n \le 60000, \qquad 1 \le k \le 1200,$$

$$1 \le a_i \le k, \qquad 1 \le h_x \le c_x \le 10^9.$$

同一输入文件内，$\sum n \le 60000$，$\sum k \le 1200$。
时间限制为 3 秒，空间限制为 256 MiB。

区间合码（orcost）

【题目描述】

一条生产线上依次放着 n 个模块，第 i 个模块的功能码为非负整数 a_i。选取两个不同位置的模块后，它们的合码为两个功能码的按位或。

对于一段至少包含两个模块的连续区间 $[x, y]$，定义它的合码代价为：

$$c(x, y) = \min_{x \le i < j \le y} (a_i \text{ OR } a_j).$$

你需要回答 q 次询问。每次给定 l, r，求完全包含在 $[l, r]$ 内、长度至少为 2 的所有连续区间的合码代价之和，即：

$$\sum_{l \le x < y \le r} c(x, y).$$

按位或运算将两个数写成二进制后逐位计算：某一位中只要至少有一个数为 1，结果的这一位就为 1，否则为 0。例如 $2 \text{ OR } 5 = 7$。

【输入格式】

从文件 **orcost.in** 中读入数据。

第一行包含两个整数 n, q，分别表示模块数和询问数。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$，表示各模块的功能码。

接下来 q 行，每行包含两个整数 l, r，表示一次询问。

【输出格式】

输出到文件 **orcost.out** 中。

共输出 q 行，每行一个整数，依次表示各次询问的答案。

【样例 1 输入】

```
4 4
2 5 6 7
1 4
1 2
2 4
3 4
```

【样例 1 输出】

```
40
7
21
7
```

【样例 1 解释】

对于第一次询问，长度为 2 的三个区间的代价均为 7；区间 $[1, 3]$、$[2, 4]$、$[1, 4]$ 的代价分别为 $6, 7, 6$。因此答案为 $7 + 7 + 7 + 6 + 7 + 6 = 40$。

【样例 2 输入】

```
4 4
0 5 5 12
1 4
2 3
2 4
1 3
```

【样例 2 输出】

```
38
5
23
15
```

【样例 2 解释】

两个模块可以有相同的功能码，但选择的位置必须不同。区间 $[2,3]$ 内的两个模块合码为 $5 \operatorname{OR} 5 = 5$。

【样例 3 输入】

```
5 2
1073741823 1073741823 1073741823 1073741823 1073741823
1 5
2 4
```

【样例 3 输出】

```
10737418230
3221225469
```

【样例 4】

见附件 **orcost/orcost4.in** 与 **orcost/orcost4.ans**。该样例满足 $n = 5000$。

【样例 5】

见附件 **orcost/orcost5.in** 与 **orcost/orcost5.ans**。该样例达到 n, q 的上界，包含交错的功能码。

【样例 6】

见附件 **orcost/orcost6.in** 与 **orcost/orcost6.ans**。该样例达到 n, q 的上界，包含多段相同的功能码。

【样例 7】

见附件 **orcost/orcost7.in** 与 **orcost/orcost7.ans**。该样例达到 n, q 的上界，所有询问均满足 $l = 1$。

【样例 8】

见附件 **orcost/orcost8.in** 与 **orcost/orcost8.ans**。该样例满足 $n = 400$，包含所有可能的询问区间。

【数据范围】

本题共 20 个测试点，每个测试点 5 分。各测试点的额外限制如下。

测试点编号	额外限制
$1 \sim 3$	$n \leq 400$
$4 \sim 7$	$n \leq 5000$
$8 \sim 11$	每次询问均满足 $l = 1$
$12 \sim 20$	无额外限制

对于所有测试点，保证 $2 \leq n \leq 50000$，$1 \leq q \leq 100000$，$0 \leq a_i < 2^{30}$，$1 \leq l < r \leq n$。
时间限制：2 秒。空间限制：256 MiB。